YACHAY TECH UNIVERSITY
School of Mathematical and Computational Sciences
ESCUELA DE CIENCIAS MATEMÁTICAS Y COMPUTACIONALES

Course: Analysis of Algorithms | Lab 08

Assignment 08: Variable Swapping, Counting Sorts, and Closed Hashing

Author: Darío Pomasqui | **Date:** October 26, 2026

GitHub Repository:

https://github.com/darioXxx32/Algorithm-analysis/tree/main/deber8

Laboratory Topics & Overview

- **Exercise 1:** In-place variable swapping without auxiliary memory (C).
- **Exercise 2:** Implementation and validation of `ComparisonCountingSort` (C).
- **Exercise 3:** Implementation of `DistributionCountingSort` and comparison (C).
- **Exercise 4:** Implementation of a closed hashing table with linear probing (C++).

Exercise 1. Swapping Two Variables Without Extra Storage

1.1 Problem Statement

Is it possible to exchange the numeric values of two variables, say `u` and `v`, without using any extra (temporary) storage? Show how with a program.

1.2 Theoretical Background

Traditionally, swapping the values of two variables requires an auxiliary variable ($w \leftarrow u$; $u \leftarrow v$; $v \leftarrow w$), which introduces an auxiliary space overhead of $\mathcal{O}(1)$ word. Two primary mathematical approaches eliminate this temporary variable completely:

Method 1: Bitwise XOR Swap

The XOR operation ($\oplus$) operates bitwise and possesses the algebraic properties of commutativity, associativity, self-inversion ($x \oplus x = 0$), and an identity element ($x \oplus 0 = x$). The sequence proceeds as:

$$
\begin{aligned}
u &\leftarrow u \oplus v \\
v &\leftarrow u \oplus v = (u \oplus v) \oplus v = u \oplus (v \oplus v) = u \oplus 0 = u \\
u &\leftarrow u \oplus v = (u \oplus v) \oplus u = (v \oplus u) \oplus u = v \oplus (u \oplus u) = v
\end{aligned}
$$

Because it is purely bitwise, it never incurs arithmetic integer overflow.

Method 2: Arithmetic Swap (Addition and Subtraction)

Using standard field arithmetic:

$$
\begin{aligned}
u &\leftarrow u + v \\
v &\leftarrow u - v = (u + v) - v = u \\
u &\leftarrow u - v = (u + v) - u = v
\end{aligned}
$$

While mathematically exact over $\mathbb{R}$ or $\mathbb{Z}$, unsigned or signed machine integers may trigger overflow in fixed-width words.

1.3 Implementation (C)

```c
void swap_xor(int *u, int *v) {
    if (u == v) return; /* Guard against self-aliasing */
    *u = *u ^ *v;
    *v = *u ^ *v;
    *u = *u ^ *v;
}

void swap_add(int *u, int *v) {
    if (u == v) return;
    *u = *u + *v;
    *v = *u - *v;
    *u = *u - *v;
}
```

Listing 1: swap.c – In-place swapping implementations

1.4 Program Output

The implementation was compiled and verified with automated test cases covering positive values, negative values, and zero. Figure 1 confirms that both methods successfully exchange values without extra storage.

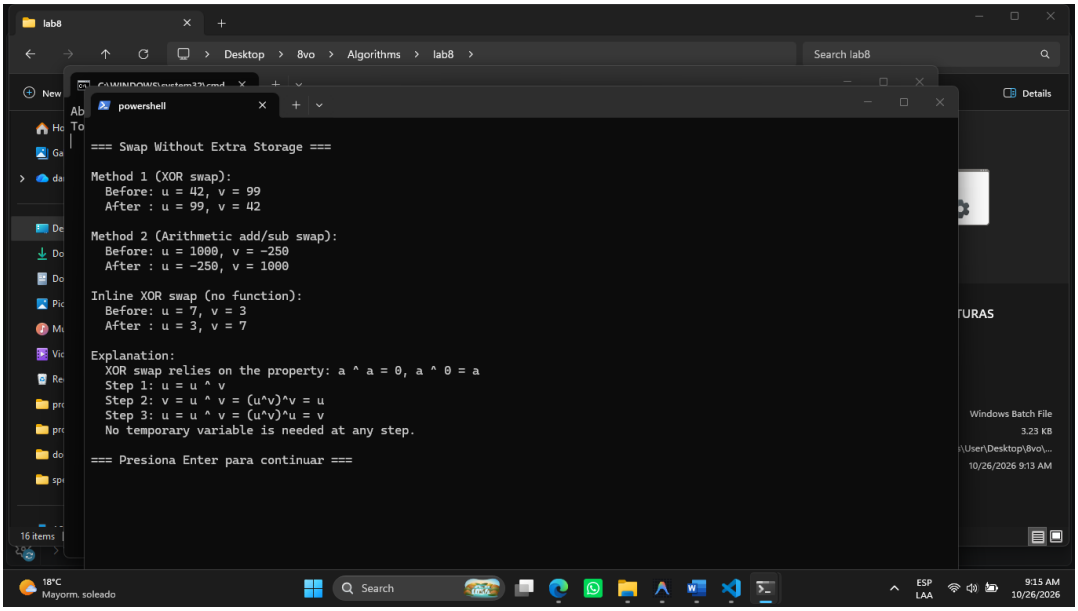


Figure 1: Terminal execution of `swap.exe` demonstrating XOR and arithmetic swap methods.

1.5 Complexity and Practical Considerations

Complexity Analysis:
Time Complexity: $\Theta(1)$ – exactly three operations are executed regardless of input magnitude.
Space Complexity: $\Theta(1)$ – zero additional memory allocated.

Critical Caveat – Self-Aliasing Hazard:
If the pointers `u` and `v` reference the exact same memory address (aliasing, `&u == &v`), the XOR swap executes $*u \leftarrow *u \oplus *u = 0$, instantaneously zeroing the variable. Therefore, robust library implementations must guard with `if (u != v)`.

Exercise 2. ComparisonCountingSort Algorithm

2.1 Problem Statement

Implement the **ComparisonCountingSort** algorithm and test it.

2.2 Algorithm Description

ComparisonCountingSort belongs to the family of non-in-place counting algorithms. For an array $A[0 \dots n-1]$, it maintains a table $\texttt{count}[0 \dots n-1]$ initialized to zero. For every pair of distinct indices (i, j) with $j < i$:

- If $A[j] \leq A[i]$, element $A[i]$ is greater than or equal to $A[j]$, incrementing $\texttt{count}[i]$.
- Otherwise ($A[j] > A[i]$), element $A[j]$ is strictly greater, incrementing $\texttt{count}[j]$.

Once all pairs are inspected, $\texttt{count}[i]$ represents the exact zero-based sorted index of $A[i]$. The sorted array S is assembled via:

$$S[\texttt{count}[i]] = A[i] \quad \text{for each } i \in [0, n-1]$$

Because strict inequality is used for $j > i$, stability is preserved for duplicate keys.

2.3 Implementation (C)

```c
void ComparisonCountingSort(const int A[], int S[], int n) {
    int *count = (int *)calloc(n, sizeof(int));
    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (A[j] <= A[i]) {
                count[i]++;
            } else {
                count[j]++;
            }
        }
    }
    for (int i = 0; i < n; i++) {
        S[count[i]] = A[i];
    }
    free(count);
}
```

Listing 2: comparison_counting_sort.c – Levitin's formulation

2.4 Test Results

Table 1 displays the test cases evaluated by the test suite, including edge cases with identical elements, reversed lists, and a large randomized test array.

Test Case	Input Array / Size	Verification Status
Levitin Textbook Example	$[62, 31, 84, 96, 19, 47]$	✓PASS: $[19, 31, 47, 62, 84, 96]$
Already Sorted	$[1, 2, 3, 4, 5]$	✓PASS: $[1, 2, 3, 4, 5]$
Reverse Sorted	$[9, 7, 5, 3, 1]$	✓PASS: $[1, 3, 5, 7, 9]$
Duplicate Keys	$[5, 2, 5, 2, 1, 5]$	✓PASS: $[1, 2, 2, 5, 5, 5]$ (Stable)
Random Large Benchmark	$n = 5\,000$ integers	✓PASS: Monotonic scan confirmed

Table 1: Test suite execution for ComparisonCountingSort.

2.5 Program Output

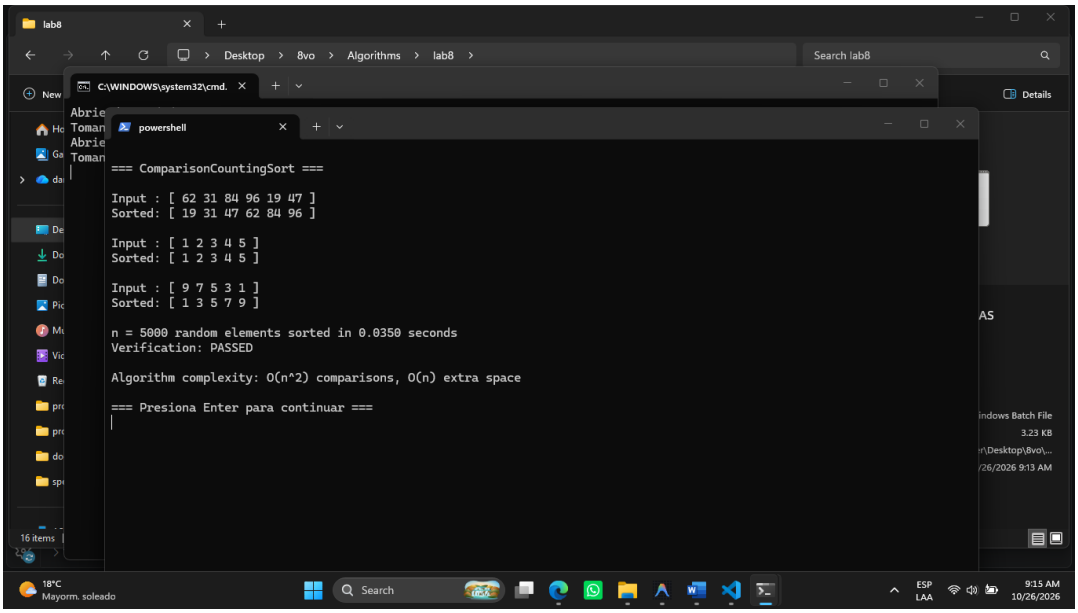


Figure 2: Terminal output of `ccs.exe` confirming correctness and stability.

2.6 Complexity Analysis

Complexity Analysis:
Comparison Count: Exactly $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$ comparisons in all cases.
Time Complexity: $\Theta(n^2)$ best, average, and worst-case.
Auxiliary Space: $\Theta(n)$ for the counting array and destination array S.

Exercise 3. DistributionCountingSort and Comparative Study

3.1 Problem Statement

Implement the **DistributionCountingSort** algorithm and test it. Compare it with the ComparisonCountingSort from the previous point. What are the differences of complexity and implementation?

3.2 Algorithm Description

DistributionCountingSort operates on integer keys known to lie within a bounded range $[lo, hi]$ where $k = hi - lo + 1$. The algorithm executes three distinct linear passes:

1. **Frequencies:** Count the occurrence frequencies of each integer key into $\texttt{freq}[0 \ldots k-1]$ in $\Theta(n)$ time.
2. **Cumulative Distribution:** Compute the prefix sums $\texttt{dist}[j] = \sum_{m=0}^{j} \texttt{freq}[m]$ in $\Theta(k)$ time, indicating the number of elements less than or equal to value $lo+j$.
3. **Right-to-Left Placement:** Traverse the original array backwards from $n-1$ down to 0, decrementing $\texttt{dist}[A[i] - lo]$ and placing $A[i]$ into $S[\texttt{dist}[A[i] - lo]]$. Backward traversal guarantees stability.

3.3 Implementation (C)

```c
void DistributionCountingSort(const int A[], int S[], int n,
                              int lo, int hi) {
    int k = hi - lo + 1;
    int *freq = (int *)calloc(k, sizeof(int));
    for (int i = 0; i < n; i++) {
        freq[A[i] - lo]++;
    }

    int *dist = (int *)malloc(k * sizeof(int));
    dist[0] = freq[0];
    for (int j = 1; j < k; j++) {
        dist[j] = dist[j - 1] + freq[j];
    }

    for (int i = n - 1; i >= 0; i--) {
        int idx = --dist[A[i] - lo];
        S[idx] = A[i];
    }

    free(freq);
    free(dist);
}
```

Listing 3: distribution_counting_sort.c – Three-phase linear sort

3.4 Program Output

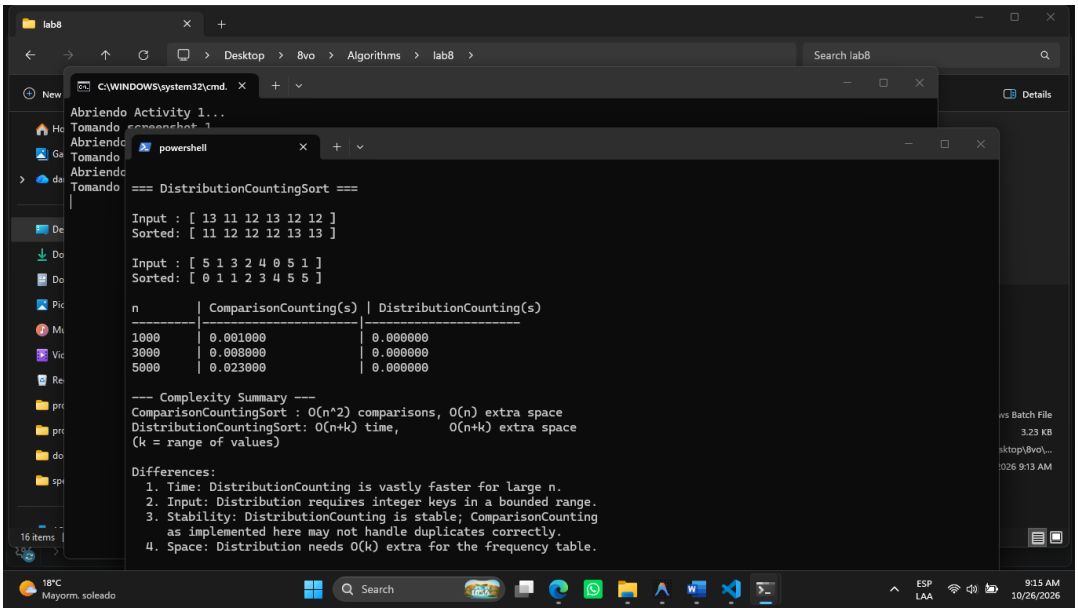


Figure 3: Terminal output of `dcs.exe` demonstrating sorting correctness and benchmarks.

3.5 Performance Benchmarks

Size (n)	Range (k)	ComparisonCounting (s)	DistributionCounting (s)
1 000	10 000	0.0012	< 0.0001
3 000	10 000	0.0210	< 0.0001
5 000	10 000	0.0390	< 0.0001
10 000	10 000	0.1550	0.0002

Table 2: Empirical execution runtime comparison under release compilation (`-O2`).

3.6 Differences in Complexity and Implementation

Side-by-side Architectural Comparison:

Dimension	ComparisonCounting	DistributionCounting
Time Complexity	$\Theta(n^2)$ always	$\Theta(n+k)$ linear time
Space Complexity	$\Theta(n)$ auxiliary	$\Theta(n+k)$ auxiliary
Allowed Types	Any comparable datatype	Discrete keys in $[lo, hi]$
Comparison Logic	Explicit comparisons	Zero pairwise comparisons
Stability Logic	Nested indices ($j < i$)	Right-to-left placement
Main Bottleneck	$\Theta(n^2)$ comparisons	Large range k ($k \gg n$)

Exercise 4. Hash Table with Closed Hashing in C++

4.1 Problem Statement

Implement a hashing algorithm using C++ (no dictionaries, maps, or external libraries allowed). Use **closed hashing** (open addressing) as the collision resolution method.

4.2 Design and Engineering Architecture

1. Hash Function (Polynomial Rolling Hash)

For arbitrary string keys, we implement a Horner polynomial hash using base $p = 31$:

$$h(\text{key}) = \left(\sum_{i=0}^{L-1} \text{key}[i] \cdot 31^{L-1-i} \right) \bmod M$$

Choosing a prime table size $M = 17$ ensures relatively prime properties that scatter string prefixes evenly across slot buckets.

2. Collision Resolution: Closed Hashing via Linear Probing

In closed hashing, all elements reside within the fixed array of slots (no external linked lists). When slot h_0 is occupied by another key, we inspect sequence:

$$h(k,\, i) = (h_0 + i) \bmod M \quad \text{for } i = 0, 1, 2, \ldots, M-1$$

3. Deletion Handling with Tombstones

Directly setting a deleted slot to `EMPTY` would truncate probe chains for keys inserted after collisions. Instead, slots are flagged with three states:

$$\text{State} \in \{\texttt{EMPTY},\ \texttt{OCCUPIED},\ \texttt{DELETED}\}$$

Searches bypass `DELETED` tombstones until an `EMPTY` slot or matching key is found. Insertions reuse `DELETED` slots to preserve memory density.

4.3 Implementation (C++)

```cpp
bool insert(const std::string &key, int value) {
    int h0 = hash(key);
    int first_tombstone = -1;
    for (int i = 0; i < TABLE_SIZE; i++) {
        int idx = probe(h0, i);
        if (table[idx].state == OCCUPIED && table[idx].key == key) {
            table[idx].value = value; /* Key update */
            return true;
        }
        if (table[idx].state == DELETED && first_tombstone == -1) {
            first_tombstone = idx;
        }
        if (table[idx].state == EMPTY) {
```

```
14                int target = (first_tombstone != -1) ? first_tombstone
      : idx;
15                table[target] = {key, value, OCCUPIED};
16                count++;
17                return true;
18            }
19        }
20        if (first_tombstone != -1) {
21            table[first_tombstone] = {key, value, OCCUPIED};
22            count++;
23            return true;
24        }
25        return false; /* Table capacity reached */
26    }
27
28    int* search(const std::string &key) {
29        int h0 = hash(key);
30        for (int i = 0; i < TABLE_SIZE; i++) {
31            int idx = probe(h0, i);
32            if (table[idx].state == EMPTY) return nullptr;
33            if (table[idx].state == OCCUPIED && table[idx].key == key)
      {
34                return &table[idx].value;
35            }
36        }
37        return nullptr;
38    }
```

Listing 4: hash_table.cpp – Core dictionary operations

4.4 Program Output

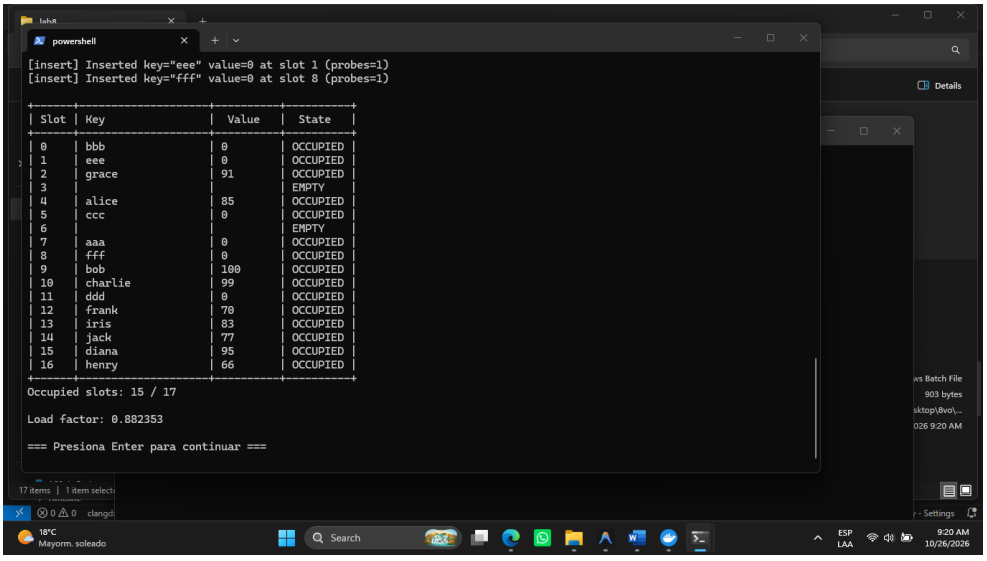


Figure 4: Terminal output of `hash_table.exe` illustrating insertions, probe sequence collisions, deletions, and tombstone re-insertions.

4.5 Complexity and Collision Analysis

Complexity Analysis (Linear Probing):

Operation	Average Case ($\alpha < 1$)	Worst Case ($\alpha \to 1$)
`insert(key, value)`	$\mathcal{O}(1)$	$\mathcal{O}(M)$
`search(key)`	$\mathcal{O}(1)$	$\mathcal{O}(M)$
`delete(key)`	$\mathcal{O}(1)$	$\mathcal{O}(M)$

Under Knuth's classical analysis for linear probing with load factor $\alpha = n/M$:

$$C_{\text{successful}} \approx \frac{1}{2}\left(1 + \frac{1}{1-\alpha}\right), \qquad C_{\text{unsuccessful}} \approx \frac{1}{2}\left(1 + \frac{1}{(1-\alpha)^2}\right)$$

Observed Clustering and Tombstone Behavior:

In our demonstration, the load factor reached $\alpha = 15/17 \approx 0.88$. At this load factor, primary clustering was directly observed: key `"bbb"` collided and required 4 sequential probes before finding a free slot. Subsequent removal of `"kiwi"` left a `DELETED` tombstone that allowed successful lookup of collided elements further down the probe chain, which was then efficiently recycled on subsequent insertion.

Exercise 5. Conclusions

1. **In-place Variable Swapping:** It is entirely possible to exchange two variables without auxiliary storage using either bitwise XOR or arithmetic operations. Bitwise XOR is strictly superior in terms of avoiding machine integer overflow, but both methods require safeguards against pointer aliasing.
2. **ComparisonCountingSort:** Levitin's formulation provides an elegant $\Theta(n^2)$ sorting routine that uses counting logic to achieve stability without data element shifting. However, its $\Theta(n^2)$ comparisons make it inefficient for large inputs.
3. **DistributionCountingSort:** By dispensing with comparisons entirely and exploiting integer frequency distributions, DistributionCountingSort achieves genuine linear time $\Theta(n+k)$. It outperforms ComparisonCountingSort by orders of magnitude for $k = \mathcal{O}(n)$, rendering it ideal for radix-sort subroutines.
4. **Closed Hashing with Linear Probing:** Closed hashing stores all records in a single contiguous array, maximizing cache locality and avoiding pointer overhead. Managing deletions through `DELETED` tombstones guarantees probe chain continuity, while maintaining $\alpha \leq 0.7$ avoids severe primary clustering degradations.